

Equazione del Calore 2D e Riordinamento nelle Fattorizzazioni Cholesky

Documento di Progetto
Informatica con Laboratorio – A.A. 2025/2026

Giosuè Aiello

1 Impostazione del problema

L'obiettivo del progetto è la risoluzione numerica dell'equazione del calore in regime stazionario in due dimensioni spaziali. Il dominio di calcolo è il quadrato unitario $\Omega = [0, 1]^2$, e si cerca la funzione di temperatura $u : \Omega \rightarrow \mathbb{R}$ che soddisfa l'equazione ellittica:

$$\kappa \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right) + f(x, y) = 0, \quad (x, y) \in \Omega, \quad (1)$$

con condizioni al contorno di Dirichlet sul bordo $\partial\Omega$:

$$u(x, y) = u_0(x, y), \quad (x, y) \in \partial\Omega.$$

I parametri fisici del problema sono: coefficiente di diffusività termica $\kappa = 0.01$ e termine sorgente $f(x, y) = e^{-10(x^2+y^2)}$, che modella una sorgente di calore concentrata in prossimità dell'origine con decadimento gaussiano. Nel caso base si considera $u_0 \equiv 0$ (condizioni omogenee).

Discretizzazione alle differenze finite e appiattimento

Il dominio continuo Ω viene sostituito da una griglia cartesiana uniforme composta da $(N + 2) \times (N + 2)$ nodi fisici, spazati tra loro di un passo costante $h = 1/(N + 1)$. I singoli nodi sono identificati dalle coordinate spaziali discrete $(x_i, y_j) = (i \cdot h, j \cdot h)$, dove gli indici spaziali i, j variano nell'intervallo $\{0, 1, \dots, N + 1\}$.

In questo reticolo si distinguono due categorie di nodi:

- **Nodi di bordo:** corrispondono agli indici $i \in \{0, N + 1\}$ oppure $j \in \{0, N + 1\}$. In questi punti la temperatura è nota a priori grazie alle condizioni di Dirichlet e non costituisce un'incognita del sistema.
- **Nodi interni:** corrispondono agli indici $1 \leq i \leq N$ e $1 \leq j \leq N$. Questi rappresentano i veri gradi di libertà del problema. Indichiamo con $u_{i,j} \approx u(x_i, y_j)$ l'approssimazione numerica della temperatura in tali punti. Essi costituiscono le N^2 incognite del nostro sistema.

Poiché i risolutori algebrici di sistemi lineari operano su array monodimensionali, è strettamente necessario “appiattare” (flattening) la griglia 2D in un vettore unidimensionale $\mathbf{u} \in \mathbb{R}^{N^2}$. A tale

scopo, introduciamo una mappa di indicizzazione biunivoca $\phi(i, j)$, definita come l'ordinamento naturale riga per riga:

$$k = \phi(i, j) = (i - 1) \cdot N + (j - 1), \quad k \in \{0, 1, \dots, N^2 - 1\}. \quad (2)$$

Tramite questa funzione, la temperatura bidimensionale $u_{i,j}$ viene mappata in modo univoco nell'elemento k -esimo del vettore delle incognite $\mathbf{u}$. Analogamente, il termine noto e i contributi noti al bordo andranno a comporre il vettore globale $\mathbf{b} \in \mathbb{R}^{N^2}$.

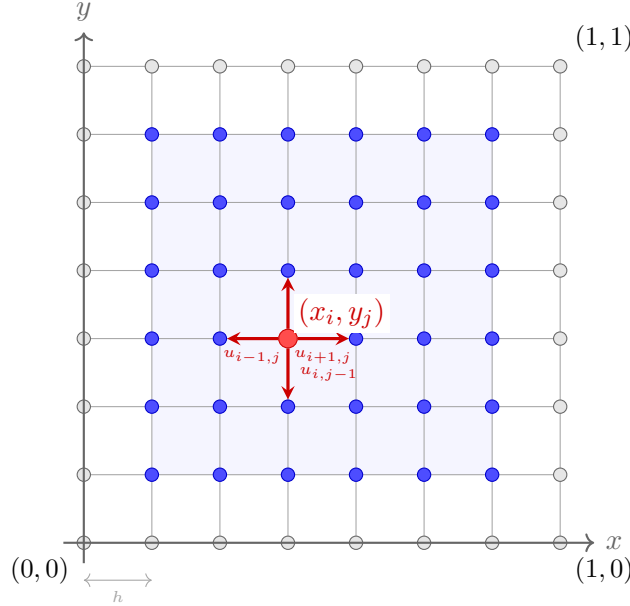


Figura 1: Griglia di discretizzazione: nodi di bordo (grigio), nodi interni (blu) e i quattro vicini dello stencil per il nodo (x_i, y_j) (frecce rosse).

Approssimando le derivate seconde con lo stencil a cinque punti, l'equazione (1) valutata nel punto interno (x_i, y_j) diventa:

$$\frac{\kappa}{h^2} (u_{i-1,j} + u_{i+1,j} + u_{i,j-1} + u_{i,j+1} - 4u_{i,j}) + f(x_i, y_j) = 0. \quad (3)$$

L'insieme delle equazioni (3), combinate tramite l'appiattimento $\phi(i, j)$, costituisce il sistema lineare algebrico globale $A\mathbf{u} = \mathbf{b}$. In questa formulazione matriciale:

- $\mathbf{u} \in \mathbb{R}^{N^2}$ è il **vettore delle incognite**, la cui componente k -esima contiene la temperatura approssimata $u_{i,j}$ nel nodo interno (x_i, y_j) mappato dall'indice k .
- $\mathbf{b} \in \mathbb{R}^{N^2}$ è il **vettore dei termini noti**, la cui componente k -esima raccoglie il termine sorgente (cambiato di segno, $-f(x_i, y_j)$) unito agli eventuali contributi noti derivanti dai nodi di bordo adiacenti al nodo (i, j) .
- $A \in \mathbb{R}^{N^2 \times N^2}$ è la **matrice dei coefficienti**, che risulta sparsa (avendo al più 5 elementi non nulli per riga, in corrispondenza del pattern dello stencil), simmetrica e definita negativa.

Fattorizzazione di Cholesky e fill-in

Poiché $-A$ è simmetrica definita positiva (SPD), si calcola la fattorizzazione di Cholesky $-A = LL^T$ con L triangolare inferiore sparsa. Durante la fattorizzazione si generano elementi

non nulli in posizioni che erano zero in $-A$: questo fenomeno è detto *fill-in* e dipende fortemente dall'ordine con cui le incognite sono enumerate.

Con l'ordinamento naturale per righe la larghezza di banda di A è $\Theta(N)$ e il fill-in di L (che determina il **costo spaziale** o occupazione di memoria) cresce come $O(N^3)$. La tecnica di *nested dissection* riduce questo costo spaziale a $O(N^2 \log N)$, con un risparmio sostanziale di memoria per valori grandi di N .

Nested dissection: approccio geometrico

La struttura di A è codificata dal *grafo di adiacenza* $G = (V, E)$: ogni nodo $v \in V$ corrisponde a una variabile $u_{i,j}$ e ogni arco $(u, v) \in E$ indica che $A_{uv} \neq 0$. Per la griglia regolare:

$$(x_i, y_j) \leftrightarrow (x_k, y_l) \iff (i = k, |j - l| = 1) \text{ oppure } (j = l, |i - k| = 1).$$

La *nested dissection* individua ricorsivamente un separatore V_S che divide V in due insiemi bilanciati V_1 e V_2 senza archi diretti tra loro. L'ordinamento risultante elenca prima V_1 , poi V_2 , infine V_S , in modo che il separatore appaia in fondo alla matrice, limitando la propagazione del fill-in.

In questo progetto si adotta un **approccio geometrico puro**: il separatore non è determinato esplorando il grafo di adiacenza, bensì è definito direttamente tramite le coordinate spaziali dei nodi. Scelto l'asse di taglio $c \in \{x, y\}$ e calcolato l'indice mediano $\hat{c}$, si pone:

$$V_1 = \{u_{i,j} \mid c < \hat{c}\}, \quad V_2 = \{u_{i,j} \mid c > \hat{c}\}, \quad V_S = \{u_{i,j} \mid c = \hat{c}\}.$$

Il procedimento viene applicato ricorsivamente a V_1 e V_2 , alternando il taglio lungo x e y . Per griglie cartesiane regolari, il piano mediano è *al contempo* un separatore geometrico e un separatore topologico del grafo: non esistono archi di adiacenza tra V_1 e V_2 (i punti sono separati da almeno un passo reticolare lungo l'asse di taglio), quindi l'approccio geometrico produce esattamente il medesimo separatore ottimale che si otterrebbe esplorando la topologia del grafo, senza il costo di tale esplorazione.

Approccio geometrico vs. approccio su grafo

Esistono due famiglie di algoritmi per la nested dissection:

1. **Approccio su grafo** (es. bisection spettrale, METIS): esplora esplicitamente il grafo di adiacenza $G = (V, E)$ e calcola autovettori del Laplaciano. Richiede $O(|E|) = O(N^2)$ solo per costruire e visitare il grafo, più il costo delle euristiche di partizionamento (generalmente $O(N^2 \alpha(N))$). Tali metodi sono necessari per griglie non strutturate o domini irregolari.
2. **Approccio geometrico** (adottato in questo progetto): ignora completamente la lista di adiacenza. Il separatore viene individuato tramite il *piano mediano* sull'asse corrente: dati gli indici interi (i, j) dei nodi, si calcola $\hat{c} = \lfloor (\min_c + \max_c)/2 \rfloor$ e si partiziona $V_S = \{v \mid c_v = \hat{c}\}$, $V_1 = \{v \mid c_v < \hat{c}\}$, $V_2 = \{v \mid c_v > \hat{c}\}$.

Perché l'approccio geometrico è superiore per griglie regolari. Per una griglia cartesiana uniforme $N \times N$ l'approccio geometrico è *equivalente* a quello su grafo in termini di qualità del separatore, ma *superiore* sotto ogni aspetto implementativo e in parte anche dei costi:

- **Correttezza del separatore:** il piano mediano $c = \hat{c}$ divide la griglia in due metà con al massimo $\lceil N/2 \rceil$ nodi ciascuna. Non esistono archi di adiacenza tra V_1 e V_2 (i nodi adiacenti si trovano a distanza reticolare h lungo l'asse di taglio, quindi il nodo con $c = \hat{c}$ funge da

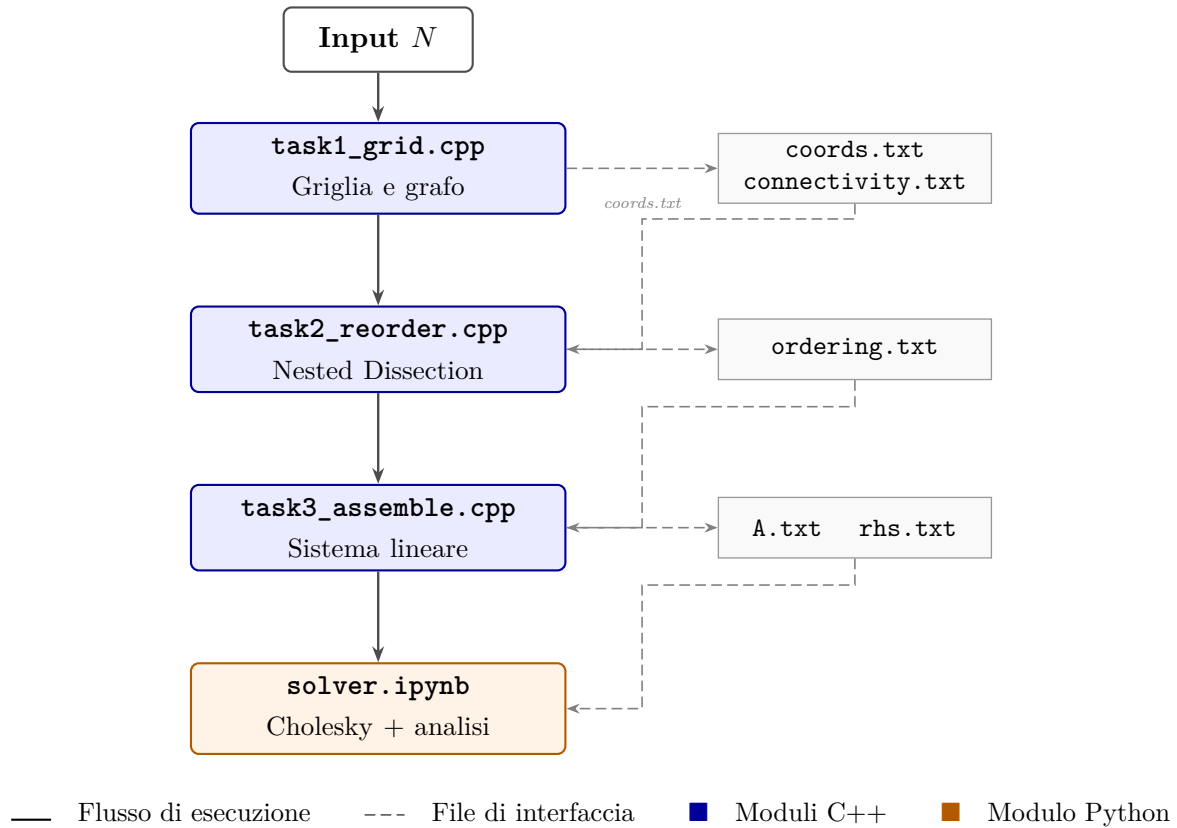
buffer). Il separatore geometrico *coincide* esattamente con il separatore topologico del grafo.

- **Complessità:** $\mathcal{O}(N^2 \log N)$ deterministico, senza euristiche. L'approccio su grafo richiede almeno $\mathcal{O}(N^2)$ per la sola lettura degli archi, più il costo dell'algoritmo di partizionamento.
- **Efficienza in cache:** accesso sequenziale al solo vettore di nodi (`coords.txt` $\rightarrow$ `std::vector<Node>`), nessuna lista di puntatori per adiacenze.
- **Robustezza:** utilizzo di confronti su indici interi $((i, j))$ anziché coordinate floating-point, eliminando i rischi di arrotondamento.

2 Architettura del progetto

Modulo principale: HES-HeatEquationSolver

Il sistema viene concepito come un unico risolutore logico denominato HES-HeatEquationSolver, articolato in quattro sotto-moduli specializzati. I tre moduli in C++ comunicano tra loro attraverso file di testo con formato fisso che fungono da interfaccia verificabile indipendentemente; il modulo Python costituisce il punto di ingresso per la fase numerica finale.



Le strutture dati fondamentali dell'intero sistema sono:

- **Liste di adiacenza** (`std::vector<std::vector<int>>`) nel Modulo 1 per la costruzione e scrittura del grafo di adiacenza.
- **Vettore di nodi logici** (`std::vector<Node>`) nel Modulo 2: `struct Node` contiene `id`, `i`, `j`; le coordinate in virgola mobile vengono lette e ignorate dal parser per minimizzare l'occupazione in cache. Questo consente la nested dissection geometrica calcolando il mediano sugli indici senza caricare in memoria la lista di adiacenza.
- **Ricorsione classica** nel Modulo 2: l'algoritmo divide-et-impera è implementato ricorsivamente come da specifiche, potendo contare su una profondità limitata dell'albero delle chiamate.
- **Vettori di permutazione** (`std::vector<int> perm, inv_perm`) per la mappatura bidirezionale tra ordinamento naturale e riordinato.
- **Formato sparso CSC** (`scipy.sparse.csc_matrix`) per la matrice A nel modulo Python, richiesto nativamente da CHOLMOD.

3 Specifiche dei moduli

I quattro sotto-moduli vengono descritti con livello di dettaglio crescente. Per ciascuno si indicano input, output, funzionalità richieste, strutture dati, invarianti, dipendenze e complessità attesa.

Modulo 1 – `task1_grid.cpp`: generazione della griglia e del grafo

Il Modulo 1 è responsabile della creazione della struttura geometrica del dominio e del relativo grafo di adiacenza. Costituisce il punto di ingresso dell'intera pipeline e non dipende da nessun altro modulo.

Formalizzazione matematica

Il dominio $\Omega = [0, 1]^2$ viene discretizzato con passo $h = \frac{1}{N+1}$. L'insieme dei nodi interni (le incognite) è:

$$V = \{(x_i, y_j) \mid x_i = i \cdot h, y_j = j \cdot h, i, j \in \{1, \dots, N\}\}, \quad |V| = N^2.$$

Il grafo di adiacenza $G = (V, E)$ ha archi:

$$E = \left\{ \{u, v\} \subset V \mid \sqrt{(x_i - x_k)^2 + (y_j - y_l)^2} = h \right\}.$$

Ogni nodo interno ha al massimo 4 vicini (sinistra, destra, basso, alto). A ogni nodo viene assegnato un identificatore intero univoco tramite la mappa $\phi(i, j) = (i - 1) \cdot N + (j - 1)$.

Specifiche

- **Input:** Intero $N > 0$ passato da riga di comando; se assente o non valido, si attiva un prompt iterativo con opzione di uscita (`q`).
- **Output:**
 - `output/coords.txt` – una riga per nodo: `id i j x y`
 - `output/connectivity.txt` – una riga per arco: `e u v` (dove `e` è l'indice progressivo dell'arco e $u < v$)
- **Funzionalità:**
 - Parsing e validazione di N con fallback interattivo a ciclo (loop).
 - Generazione dei N^2 nodi con ID progressivo e coordinate spaziali (precisione `double`).
 - Costruzione della lista di adiacenza bidirezionale e scrittura degli archi senza duplicati ($u < v$).
- **Strutture dati:** `std::vector<std::vector<int>>` di dimensione N^2 , pre-allocata con `reserve(4)` per ciascun nodo.
- **Invarianti:**
 - La lista di adiacenza è sempre simmetrica: se $v \in \text{adj}[u]$ allora $u \in \text{adj}[v]$.
 - Ogni arco appare esattamente una volta in `connectivity.txt` (condizione $u < v$).
 - Il numero di righe in `coords.txt` è esattamente N^2 .
- **Dipendenze:** Nessuna. Questo modulo è il primo della pipeline.
- **Complessità attesa:** $\mathcal{O}(N^2)$ in tempo e spazio.

Logica implementativa (pseudo-codice)

```
1 Programma Principale (Generazione Griglia):
2   -- Inizializzazione --
3   Leggi la Dimensione_Griglia (N) passata dall'utente
4   Se la dimensione non e' valida:
5       Richiedi l'inserimento ciclicamente finche' non e' un numero positivo
6
7   -- Creazione dei Nodi e degli Archi --
8   Per ogni Riga (i) da 1 fino a N:
9       Per ogni Colonna (j) da 1 fino a N:
10          Calcola l'Identificativo_Univoco del nodo attuale
11          Calcola le coordinate spaziali reali (X, Y) del nodo
12
13          Salva i dati del nodo (ID, riga, colonna, X, Y) nel file
14              'coords.txt'
15
16   -- Connessioni (Grafo) --
17   Se il nodo non e' sull'ultimo bordo destro:
18       Crea un arco di collegamento con il nodo successivo a destra
19   Se il nodo non e' sull'ultimo bordo inferiore:
20       Crea un arco di collegamento con il nodo sottostante
21
22   -- Salvataggio della Connettivita' --
23   Per ogni nodo e per ogni suo collegamento:
24       Se non e' un duplicato (evita andata e ritorno):
25           Salva il collegamento nel file 'connectivity.txt'
```

Modulo 2 – `task2_reorder.cpp`: nested dissection geometrica

Il Modulo 2 costituisce il cuore algoritmico del progetto. Calcola una permutazione ottimizzata dei nodi tramite la *nested dissection geometrica*, al fine di minimizzare il fill-in durante la fattorizzazione di Cholesky.

Formalizzazione matematica

L'algoritmo riceve in input il vettore S di nodi con coordinate intere (i, j) e reali (x, y) , e individua ricorsivamente:

$$V_1 = \{u \in S \mid c_u < \hat{c}\}$$

$$V_2 = \{u \in S \mid c_u > \hat{c}\}$$

$$V_S = \{u \in S \mid c_u = \hat{c}\}$$

dove $c \in \{i, j\}$ è l'indice dell'asse di taglio e $\hat{c} = \lfloor (\min_{u \in S} c_u + \max_{u \in S} c_u) / 2 \rfloor$. L'ordinamento risultante è $V_1 \rightarrow V_2 \rightarrow V_S$, ricorsivo con alternanza degli assi.

Specifiche

- **Input:** `output/coords.txt` (prodotto dal Modulo 1). Il file `connectivity.txt` *non viene letto*: per l'approccio geometrico la struttura topologica del grafo è ridondante.
- **Output:** `output/ordering.txt` – una riga per posizione: `new_id old_id`
- **Funzionalità:**
 - Lettura delle sole coordinate da `coords.txt`.
 - Partizionamento tramite ricorsione classica (funzione che richiama se stessa).
 - Calcolo del piano mediano con indici interi, con alternanza degli assi i/j ad ogni livello.
 - Verifica dell'invariante: `ordering.size() == N2`.
 - Salvataggio dell'ordinamento su `ordering.txt`.
- **Strutture dati:** `struct Node {id, i, j}; std::vector<int> ordering.`
- **Invarianti:**
 - Il vettore `ordering` è una permutazione valida di $\{0, \dots, N^2 - 1\}$.
 - I nodi di V_S compaiono sempre *dopo* i nodi di V_1 e V_2 alla stessa profondità.
 - L'asse di taglio si alterna ad ogni livello ($0 = i, 1 = j$).
- **Dipendenze:** Modulo 1 (solo `coords.txt`).
- **Complessità attesa:** $\mathcal{O}(N^2 \log N)$ in tempo; $\mathcal{O}(N^2)$ in spazio.

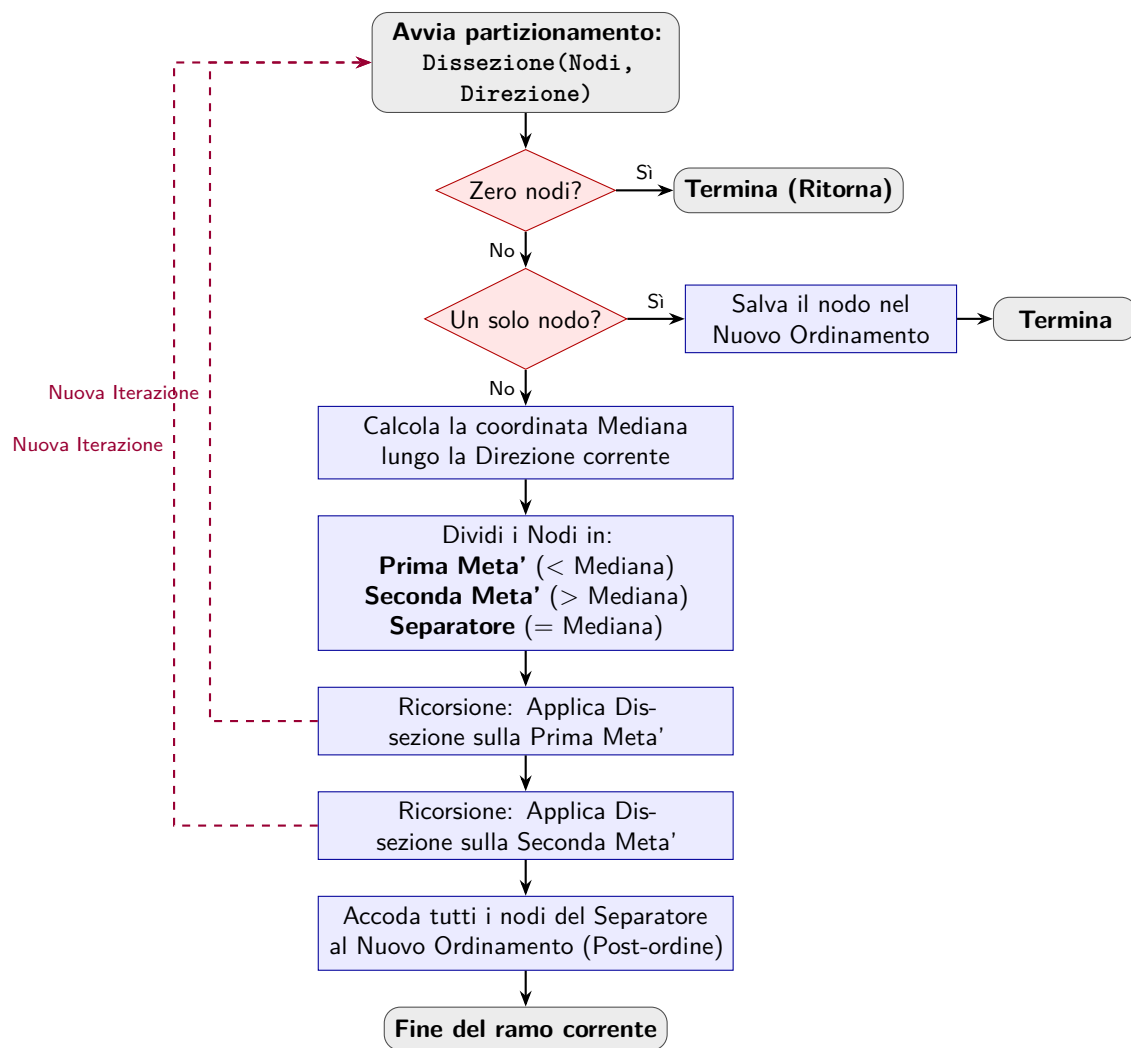


Figura 2: Diagramma di flusso (task2).



Figura 3: Esempio di ordinamento su griglia.

Logica implementativa (pseudo-codice)

```
1 Programma Principale:
2   1. Carica l'elenco dei nodi dal file 'coords.txt'
3   2. Avvia la partizione: Nested_Dissection(Tutti_i_nodi,
4       Asse_Orizzontale)
5   3. Verifica che tutti i nodi siano stati riordinati correttamente
6   4. Salva la permutazione finale nel file 'ordering.txt'
7
8 Procedura Ricorsiva Nested_Dissection(Sottoinsieme_Nodi, Asse_Corrente):
9   -- Casi base (condizioni di arresto) --
10  Se il Sottoinsieme_Nodi e' vuoto:
11    Interrompi e ritorna
12  Se il Sottoinsieme_Nodi contiene un solo nodo:
13    Aggiungi l'identificativo del nodo al nuovo ordinamento
14    Interrompi e ritorna
15
16  -- Individuazione del separatore --
17  Trova la coordinata minima e massima lungo l'Asse_Corrente
18  Calcola il Punto_Medio tra la minima e la massima
19
20  -- Partizionamento geometrico --
21  Suddividi i nodi in tre nuovi raggruppamenti:
22    - Prima_Meta (nodi con coordinata minore del Punto_Medio)
23    - Seconda_Meta (nodi con coordinata maggiore del Punto_Medio)
24    - Separatore (nodi posizionati esattamente sul Punto_Medio)
25
26  -- Alternanza della direzione di taglio (da x a y, e viceversa) --
27  Prossimo_Asse = Alterna_Direzione(Asse_Corrente)
28
29  -- Esplorazione ricorsiva (Divide et Impera) --
30  Richiama Nested_Dissection(Prima_Meta, Prossimo_Asse)
31  Richiama Nested_Dissection(Seconda_Meta, Prossimo_Asse)
32
33  -- Post-ordine (fill-in minimizzato) --
34  Per ogni nodo contenuto nel Separatore:
35    Aggiungi l'identificativo del nodo al nuovo ordinamento
```

Modulo 3 – task3_assemble.cpp: assemblaggio del sistema lineare

Il Modulo 3 costruisce il sistema lineare $Au = b$ che approssima l'equazione differenziale. Supporta sia l'ordinamento naturale che quello ottimizzato via il flag `-r`, e implementa la parte opzionale delle condizioni al bordo non omogenee.

Formalizzazione matematica

Applicando lo stencil a 5 punti con passo $h = 1/(N + 1)$ e diffusività κ , i coefficienti della matrice per ogni nodo k (nel nuovo ordinamento) sono:

$$\begin{aligned} A_{k,k} &= -\frac{4\kappa}{h^2} \\ A_{k,k'} &= +\frac{\kappa}{h^2} \quad \text{per ogni vicino interno } k' \end{aligned}$$

Il termine noto è $b_k = -f(x_{i(k)}, y_{j(k)})$. Se il vicino k' cade sul bordo (condizione di Dirichlet), il suo valore u_0 è noto e viene incorporato nel termine noto:

$$b_k \leftarrow \frac{\kappa}{h^2} \cdot u_0(x_{\text{bordo}}, y_{\text{bordo}}).$$

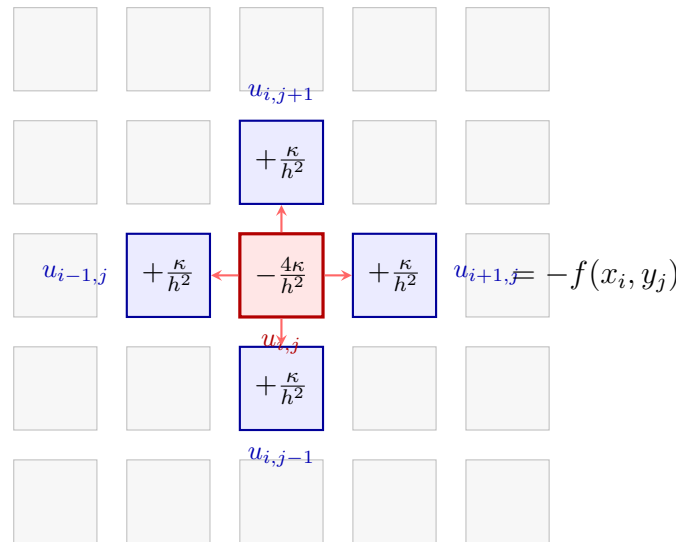


Figura 4: Maschera dello stencil a 5 punti: bilancio dell'equazione differenziale per l'assemblaggio di una riga di A .

Specifiche

- **Input:** Intero N e flag opzionale `-r` da riga di comando; `output/coords.txt`; `output/ordering.txt` (solo se `-r`).
- **Output:** `output/A.txt` (triplette `i j val`) e `output/rhs.txt` (una riga per valore di b).
- **Funzionalità:**
 - Lettura di `coords.txt` e costruzione della mappa $\phi^{-1} : (i, j) \rightarrow \text{id}$.
 - Se `-r`: lettura di `ordering.txt` e costruzione di `perm` e `inv_perm`; altrimenti vettori identità.

- Menu interattivo per la scelta della condizione al bordo u_0 (selezione avviene una sola volta, prima del ciclo di assemblaggio).
- Assemblaggio dello stencil con gestione delle condizioni di Dirichlet.
- Scrittura di `A.txt` e `rhs.txt` con precisione floating-point a 10 cifre decimali.
- **Strutture dati:** `std::vector<int> perm, inv_perm` di dimensione N^2 ; `struct Triplet {int row, col; double val}; std::vector<Triplet>` pre-allocato con `reserve(5*N*N)`; `std::vector<double> b`.
- **Invarianti:**
 - `perm` e `inv_perm` sono permutazioni inverse l'una dell'altra: `inv_perm[perm[k]] == k` per ogni k .
 - La matrice A è simmetrica: se la tripletta (k, k', v) è presente, lo è anche (k', k, v) .
 - Il numero di righe in `rhs.txt` è esattamente N^2 .
- **Dipendenze:** Modulo 1 (`coords.txt`); Modulo 2 (`ordering.txt`, solo se `-r`).
- **Complessità attesa:** $\mathcal{O}(N^2)$ in tempo (al più 5 entrate per nodo) e spazio.

Logica implementativa (pseudo-codice)

```

1 Programma Principale (Costruzione del Sistema Lineare):
2   -- Inizializzazione --
3   Leggi i parametri di avvio: Dimensione Griglia, Uso del Riordinamento,
4   Tipo Condizioni Bordo
5   Carica in memoria tutte le informazioni sui nodi create dal Modulo 1
6
7   -- Configurazione dell'Ordine --
8   Se e' attivo il Riordinamento (Nested Dissection):
9     Carica il Nuovo Ordinamento e costruisci le mappe di conversione
10    (Vecchia <-> Nuova posizione)
11  Altrimenti:
12    Usa l'ordinamento classico riga per riga
13
14  -- Costruzione delle Equazioni --
15  Per ogni riga della matrice globale (che corrisponde alla Nuova
16  Posizione del nodo):
17    Individua chi era originariamente questo nodo e recupera le sue
18    coordinate (X, Y)
19
20    Salva il termine centrale dell'equazione (l'interazione del nodo con
21    se stesso)
22    Calcola il calore generato internamente (f(x,y)) e mettilo nel
23    vettore dei termini noti
24
25  -- Interazioni con i vicini --
26  Per ogni direzione possibile (Sopra, Sotto, Sinistra, Destra):
27    Calcola la posizione del nodo vicino in quella direzione
28
29    Se il vicino esce dal bordo della griglia:
30      Applica la temperatura del bordo (Condizione di Dirichlet)
31      spostandola nel termine noto
32    Altrimenti (il vicino e' interno alla griglia):
33      Scopri qual e' la Nuova Posizione di questo vicino
34      Salva il termine di interazione termica tra il nodo attuale e
35      questo vicino
36
37  -- Salvataggio --
38  Salva tutte le interazioni calcolate nel file 'A.txt'
39  Salva tutti i termini noti nel file 'rhs.txt'

```

Condizioni al bordo disponibili (parte opzionale)

Scelta	Funzione $u_0(x, y)$	Utilità di verifica
0	0	Caso base omogeneo (default)
1	10	Con $f = 0$, la soluzione deve essere $\equiv 10$
2	x	Con $f = 0$, la soluzione interna deve coincidere con x
3	$\sin(\pi x)$	Bordo continuo; si annulla agli angoli
4	$x^2 - y^2$	$\nabla^2(x^2 - y^2) = 0$: soluzione numerica esatta
5	$\mathbf{1}_{y=1}$	Shock termico: solo il bordo superiore è caldo

Modulo 4 – `solver.ipynb`: risoluzione (Task 4) e analisi (Task 5)

Il Modulo 4 è il punto terminale della pipeline. In termini di specifiche progettuali, questo modulo accorpa coerentemente sia le richieste della **Task 4** (costruzione della matrice sparsa CSC e fattorizzazione di Cholesky) sia della **Task 5** (benchmark per $N \in \{32, \dots, 1024\}$, *spy plot* della struttura e grafici della soluzione).

Si è optato per questa scelta architetturale poiché entrambe le task si collocano naturalmente nello stesso ambiente di esecuzione (Python). Piuttosto che frammentare la logica in script separati, un unico Jupyter Notebook (`solver.ipynb`) funge da orchestratore finale: legge i dati generati dal Modulo 3, esegue i calcoli numerici e restituisce contestualmente l'analisi visiva e prestazionale.

Specifiche

- **Input:**
 - `output/A.txt` e `output/rhs.txt` (prodotti dal Modulo 3).
 - Parametro N (o serie $N \in \{32, 64, 128, 256, 512, 1024\}$ per il benchmark).
- **Output:**
 - Vettore soluzione $u \in \mathbb{R}^{N^2}$, rimappato sulla griglia originale per la visualizzazione.
 - Spy plot della matrice A e del fattore L per ordinamento naturale e nested dissection.
 - Mappa di calore 2D e superficie 3D della soluzione $u(x, y)$.
 - Grafico log-log del fill-in `nnz(L)` e dei tempi di fattorizzazione al variare di N .
- **Funzionalità:**
 - Lettura di `A.txt` e costruzione di `scipy.sparse.coo_matrix`, con conversione in `csc_matrix`.
 - Fattorizzazione $-A = LL^T$ tramite `sksparse.cholmod.cholesky` con `order="natural"`.
 - Risoluzione del sistema con `scipy.sparse.linalg.spsolve_triangular` (forward e backward substitution).
 - Automazione dell'intera pipeline C++ tramite `subprocess` per ciascun valore di N e ordinamento.
 - Misurazione dei tempi con `time.perf_counter`.
- **Strutture dati:**
 - `scipy.sparse.coo_matrix` per l'assemblaggio iniziale; `csc_matrix` per la fattorizzazione.
 - `numpy.ndarray` per la soluzione u e il termine noto b .
 - Oggetto `Factor` di `CHOLMOD` per estrarre la matrice L esplicita.
- **Invarianti:**
 - La matrice costruita da `coo_matrix` deve essere simmetrica e definita negativa.
 - Il vettore soluzione u deve soddisfare $\|Au - b\|_2 / \|b\|_2 < 10^{-10}$ (residuo relativo). Questa severa soglia assicura l'assenza di cancellazioni catastrofiche prodotte dall'accumulo

di errori in virgola mobile, tollerando al contempo il fisiologico malcondizionamento $\mathcal{O}(N^2)$ della matrice Laplaciana all'aumentare di N .

- **Dipendenze:** Modulo 3 (A.txt, rhs.txt); librerie `scipy`, `numpy`, `scikit-sparse`, `matplotlib`.
- **Complessità attesa:** Fill-in $\mathcal{O}(N^3)$ con ordinamento naturale; $\mathcal{O}(N^2 \log N)$ con nested dissection. Tempo di fattorizzazione dominato dal fill-in.

Logica implementativa (pseudo-codice)

```

1 Programma Principale (Orchestratore in Python):
2
3 -- FASE 1: Risoluzione e Validazione Fisica (es. N=32) --
4 Avvia pipeline C++ (Modulo 1, Modulo 3)
5 Leggi la matrice sparsa A da 'A.txt' e il termine noto b da 'rhs.txt'
6 Fattorizza A = L L^T (Cholesky)
7 Risolvi il sistema lineare (Sostituzione in Discesa e in Risalita)
8 Costruisci la griglia 2D includendo le condizioni al bordo
9 Mostra la mappa di calore e la superficie 3D delle temperature
10
11 -- FASE 2: Benchmark Prestazionale Asintotico --
12 Per ogni Dimensione N da testare (32, 64, 128, ...):
13   Per ogni Ordinamento (Naturale vs Nested Dissection):
14
15     Avvia pipeline C++ per lo specifico N e Ordinamento
16     Leggi ESCLUSIVAMENTE la matrice A da 'A.txt'
17
18     Fai partire il Cronometro
19     Fattorizza A = L L^T (Cholesky)
20     Ferma il Cronometro
21
22     Registra il Tempo di Fattorizzazione e conta il Fill-in (Numeri
23       Diversi da Zero in L)
24
25 -- Visualizzazione Finale Benchmark --
26 Disegna i grafici comparativi Log-Log (Tempo e Fill-in rispetto a N^2)

```

La sostenibilità dell'intera pipeline di calcolo si regge sull'efficienza dei tre moduli preprocessori scritti in C++. Affinché il solutore numerico Python finale possa gestire griglie estremamente dense (come $N = 1024$), è imperativo che la costruzione del sistema lineare e il calcolo del riordinamento non costituiscano essi stessi un collo di bottiglia.

3.1 Il collo di bottiglia dell'Ordinamento Naturale

L'approccio ingenuo (ordinamento naturale lessicografico) fallisce irrimediabilmente per esaurimento della memoria (OOM - Out of Memory) a dimensioni elevate. Poiché la larghezza di banda della matrice A è pari a N , il fattore di Cholesky L soffre di un fill-in (riempimento di zeri con valori non nulli) proporzionale a:

$$\text{Fill-in (Naturale)} = \mathcal{O}(N^3) \quad (4)$$

Per $N = 1024$, il numero di incognite è $V \approx 10^6$. Il fattore L naturale richiederebbe di immagazzinare $\approx 10^9$ elementi in virgola mobile a doppia precisione (circa 8 GB di pura RAM solo per la matrice, ignorando l'overhead), rendendo la fattorizzazione instabile o impossibile

sui computer portatili standard. La *Nested Dissection* riduce drasticamente questo fill-in a $\mathcal{O}(N^2 \log N)$, rendendo il problema computazionalmente trattabile.

La tabella seguente riassume la complessità asintotica in Spazio e Tempo dei nostri algoritmi C++ (con $V = N^2$ incognite totali):

Modulo (C++)	Complessità Temporale	Complessità Spaziale
task1_grid	$\mathcal{O}(N^2)$	$\mathcal{O}(N^2)$
task2_reorder	$\mathcal{O}(N^2 \log N)$	$\mathcal{O}(N^2)$
task3_assemble	$\mathcal{O}(N^2)$	$\mathcal{O}(N^2)$

3.2 Modulo 1: Generazione della Topologia in $\mathcal{O}(N^2)$

Il problema fisico è impostato su un dominio quadrato bidimensionale discretizzato con una griglia cartesiana. Le incognite numeriche (i gradi di libertà interni) sono esattamente $V = N^2$.

L'algoritmo di instradamento effettua una singola scansione spaziale tramite due cicli `for` annidati (iterando lungo gli assi y e x). Ad ogni iterazione, l'elaboratore esegue un set costante di operazioni logiche con costo $\mathcal{O}(1)$:

- Calcolo dell'identificativo unidimensionale `id`.
- Assegnazione delle coordinate reali x e y .
- Mappatura dei vicini (creazione degli archi).

Poiché vengono eseguiti passaggi in tempo costante per ciascuno degli N^2 nodi, il tempo globale scala linearmente con il numero totale di incognite, $\mathcal{O}(N^2)$. Lo spazio in memoria RAM è dominato dall'allocazione delle strutture dati primarie (come `std::vector<Node>`), dimensionate esattamente a N^2 istanze. Ciò garantisce una limitazione spaziale rigorosa e stabile a $\mathcal{O}(N^2)$.

3.3 Modulo 2: Nested Dissection Geometrica in $\mathcal{O}(N^2 \log N)$

Il modulo di riordinamento sfrutta la disposizione geometrica della griglia anziché esplorare il grafo. Adotta un approccio ricorsivo di tipo *Divide et Impera* per dividere il dominio in sotto-problemi.

Questa suddivisione genera un albero di decisione con profondità logaritmica pari a $\mathcal{O}(\log V)$, dove $V = N^2$ è il numero totale di incognite. Poiché a ogni livello dell'albero il lavoro complessivo svolto su tutti i sotto-problemi è proporzionale a V , la complessità temporale complessiva risulta pari a:

$$T(V) = \mathcal{O}(V \log V) \implies \mathcal{O}(N^2 \log(N^2)) \implies \mathcal{O}(N^2 \log N) \quad (5)$$

Per quanto riguarda la memoria, la ricorsione viene implementata con funzioni standard C++. L'occupazione di memoria totale per il riordinamento rimane quindi limitata a $\mathcal{O}(N^2)$.

3.4 Modulo 3: Assemblaggio del Sistema in $\mathcal{O}(N^2)$

La discretizzazione differenziale si basa su uno stencil discreto di Laplace a 5 punti. Il cuore matematico è attraversato serialmente sulle N^2 righe ideali che comporranno la matrice finale di Poisson.

Per la struttura fisica strettamente spaziale intrinseca all'operatore di Laplace bidimensionale ∇^2 , ciascuna molecola discreta si accoppia e interagisce analiticamente solo coi suoi diretti vicini (Nord, Sud, Est, Ovest). Il calcolatore impiega calcoli in esecuzione rapida $\mathcal{O}(1)$ per derivare i divisori spaziali κ/h^2 e incorporare con il segno opposto i contributi asimmetrici delle condizioni di Dirichlet al bordo $u_0(x, y)$.

Questa metodologia partorisce una matrice *sparsa* che assicura a livello algebrico la genesi di non più di 5 elementi non nulli per ciascuna riga del dominio. I moduli precludono alla radice l'inizializzazione di pesanti matrici dense temporanee (evitando allocazioni inutili di zeri); ragion per cui il tempo di saturazione dell'assemblaggio evolve in proporzione esatta a V , ovvero sia $\mathcal{O}(N^2)$. Infine, per abbattere criticamente i costi accessori dettati dalle riallocazioni dinamiche in volo, il vettore di triplette (formato COO) proprio della *Standard Library* del C++ pre-riserva la dimensione necessaria impiegando la macro-chiamata `reserve(5*N*N)`. Questa istruzione chirurgica inchioda l'impronta di RAM a una mole rigidamente stimata in $\mathcal{O}(N^2)$.

4 Piano di validazione sperimentale

L'obiettivo di questa fase di validazione architetturale è garantire la totale solidità dello scambio dati tra il backend C++ e l'interfaccia Python, nonché assicurare che la formulazione numerica del sistema lineare rispetti rigorosamente i principi fisici del problema. L'analisi approfondita delle performance computazionali (riduzione del fill-in e tempi di esecuzione) è invece demandata e ampiamente discussa all'interno del Jupyter Notebook.

1. Sanity Checks Fisici (Verifica della Soluzione)

Il modo più efficace per validare l'intero stack (generazione della griglia, assemblaggio e risoluzione) consiste nel testare il sistema con scenari fisici elementari di cui si conosce a priori la soluzione analitica. Al fine di automatizzare questa batteria di test tramite la pipeline Python, **le condizioni al bordo $u_0(x, y)$ vengono lette dinamicamente tramite `stdin`** nel modulo `task3_assemble.cpp`. Questo espediente permette di validare in modo istantaneo i seguenti scenari:

- **Il "Caso Zero" (Opzione 0):** Impostando la sorgente interna $f = 0$ e la temperatura al bordo $u_0 = 0$, la soluzione numerica calcolata deve risultare identica a zero in ogni punto del dominio. L'aderenza a questo test esclude inequivocabilmente la presenza di perdite logiche nell'assemblaggio del termine noto b .
- **Caso Temperatura Costante (Opzione 1):** Fissando uniformemente l'involucro a una costante termica C (es. $u_0 = 10$), in perfetta assenza di gradienti termici interni, la soluzione calcolata deve convergere univocamente a C su tutto il dominio.
- **Caso Funzione Armonica (Opzione 4):** Attivando la condizione armonica quadratica $u_0 = x^2 - y^2$ con $f = 0$, la soluzione numerica è forzata a ricalcare la medesima forma originaria in virtù del teorema della divergenza ($\nabla^2(x^2 - y^2) = 0$). Testando questa configurazione, lo scarto residuo della soluzione numerica si mantiene confinato nell'ordine della precisione di macchina ($\approx 10^{-16}$), confermando empiricamente la perfetta validità e accuratezza dello stencil a 5 punti.

2. Validazione e Integrità dello Scambio Dati

Poiché l'architettura sfrutta esplicitamente il file system come veicolo di comunicazione tra i layer, è essenziale prevenire qualsiasi forma di corruzione dei dati in transito:

- **Precisione Matematica Flottante in transito:** Per sventare l'introduzione di errori di arrotondamento durante il salvataggio dei coefficienti del sistema lineare, i flussi di scrittura `std::ofstream` nel C++ vengono rigorosamente forzati a una risoluzione nominale elevata tramite `std::setprecision(10)`. In questo modo, il risolutore in Python eredita una rappresentazione fedele e non degradata del Laplaciano discreto e del termine noto, mantenendo intatte le proprietà della matrice (in primis la simmetria e la definitezza positiva).